**A**

**Capstone Project Synopsis**

**On**

# "Digital Twin Personality Assistant Using Conversational AI"

**Submitted during 6th semester in partial fulfilment of the requirements for the award of degree of**

**Bachelor of Technology**

**in**

**Computer Engineering(Spec. in Data Science)**

**by**


**Keshav Kumar : 23001016029**

**Piyush : 23001016046**

**Under supervision of**

**Dr. Deepika**

**Department of Computer Engineering**

**FACULTY OF INFORMATICS & COMPUTING**

**J.C. Bose University of Science & Technology, YMCA**

**Faridabad – 121006**


**Jan-May 2026**

**TABLE OF CONTENTS**

1. **Chapter 1: Introduction**
    - **1.1 Brief of the Project**
    - **1.2 Technologies Used**
2. **Chapter 2: Proposed Methodology**
3. **Chapter 3: Applications and Scope**

# CHAPTER 1

# INTRODUCTION

## 1.1 Brief of the Project

This project proposes a **Digital Twin Personality Assistant**: a conversational agent that imitates a specific person's speech and decision-making style. By analyzing that person's historical chat data (e.g. WhatsApp conversations), the system learns their unique language use, slang, emoji habits, and behavioral patterns. The goal is to enable an AI assistant (initially as a Telegram bot forwarding to WhatsApp) to reply on behalf of the person in a natural, personalized way. This merges AI with human psychology: the assistant models the user's preferences and tendencies so it can make similar conversational "decisions." In AI research, such *human digital twins* have been envisioned as personalized agents simulating an individual's cognitive and social characteristics[1]. Large language models are increasingly seen as capable of encoding an individual's "communication style, behavioral tendencies, and personality traits"[2]. Likewise, personalized chatbots aim to have a **consistent personality** that behaves like the real user[3]. Our assistant will leverage these ideas: it combines data-driven modeling of human behavior with LLM-based chat generation. Prior work (e.g. Ma et al., 2021) shows that learning an implicit user profile from a person's dialogue history can enable a chatbot to give "personalized responses" that reflect the user's persona[3]. Similarly, recent proposals for AI-powered "digital twins" emphasize *predictive decision-making* and adaptive personalization from user data[4][1].

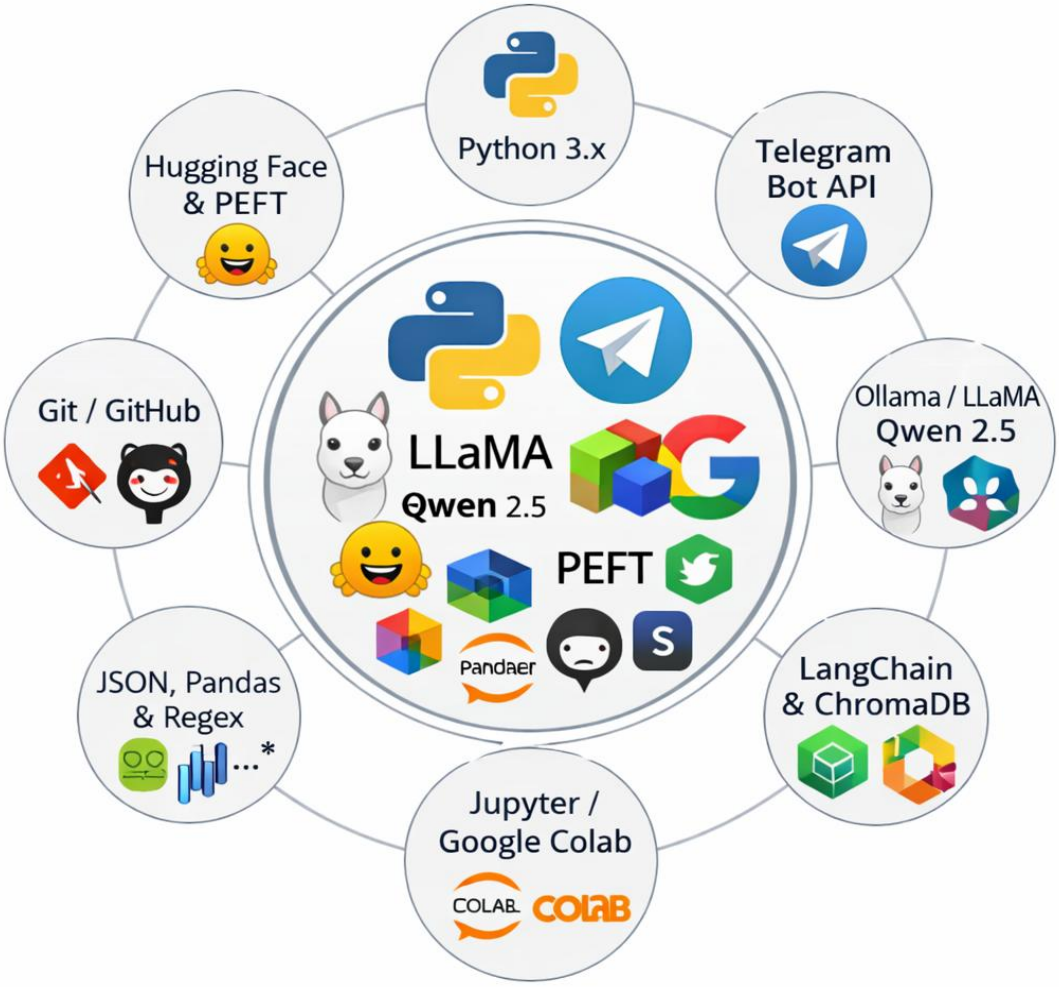
So far, we have collected and preprocessed roughly one year's worth of WhatsApp chat logs. We extracted speaker-tagged message histories to form context–reply pairs. An initial Retrieval-Augmented Generation (RAG) pipeline was implemented: conversational contexts were embedded into a vector database, and similar past contexts are retrieved to guide responses. Preliminary tests showed that simple RAG with short chats was not fully capturing the user's style, motivating the current approach of directly fine-tuning a model on the chat data (see Chapter 2).

## 1.2 Technologies Used

- **Python 3.x** – Primary programming language for data processing and modeling.
- **Telegram Bot API** – For building the bot interface to receive and send messages.
- **HuggingFace Transformers & PEFT** – Fine-tuning large language models with Low-Rank Adaptation (LoRA)[5][6].
- **Ollama (LLaMA/Gemini models like Qwen 2.5)** – Local LLM deployment for inference and generation.
- **LangChain, ChromaDB, Sentence-Transformers** – Tools for embedding user queries and performing similarity search (RAG pipeline).
- **Jupyter Notebook / Google Colab** – Interactive development and training environment (with GPU support for LoRA).
- **JSON, pandas, Regex** – For parsing WhatsApp exports and structuring datasets.
- **Git/GitHub** – Version control for code and data.

Each technology is chosen for its strength in the pipeline: LangChain and ChromaDB for efficient similarity search of past chats; Ollama with Qwen/Gemini for fast local LLM inference; Transformers/PEFT for parameter-efficient fine-tuning; and Telegram Bot API for a real user interface.

---



# CHAPTER 2

## PROPOSED METHODOLOGY

Our methodology comprises **data processing, retrieval-memory setup, and LLM training** steps:

- **Data Preparation:** We exported the user's WhatsApp chats and labeled messages by sender. We then built context–reply pairs by pairing each message from *others* (context) with the user's subsequent reply. To capture some context, we also consider the previous one or two messages as part of the context window. The data was cleaned (removing system messages, short interjections, etc.) to improve quality.

- **Retrieval-Augmented Pipeline (Prototype):** We first implemented a RAG-based approach. All (multi-turn) contexts were embedded using a sentence transformer model and stored in a Chroma vector database. At runtime, a new incoming message is converted to an embedding and used to query the DB for the most similar past contexts. We retrieve the top-k contexts and extract the user's replies to those contexts from history. These few example replies are then appended to a prompt for the LLM. This mimics the idea of feeding the model sample user replies as "in-context examples," encouraging the LLM to generate a response in the user's style. (In generation experiments, RAG has been found to yield **more specific, diverse, and factual** outputs than a non-retrieval model[7], which partly motivates using memory retrieval.)

- *Example:* If the chat history contains "**Friend:** kal clg hai? **User:** haan bhai aaunga," then for the query "kal clg hai?", the system can retrieve that context and feed "Reply: haan bhai aaunga" as an example. The LLM then aims to produce a similar casual reply.

However, our tests showed that with very short chat messages and slang, the simple RAG method struggled to produce natural answers consistently. Therefore, we pivoted to the current approach of model fine-tuning.

- **Model Fine-Tuning (LoRA):** We next prepare a fine-tuning dataset from the chat logs. The dialog pairs are converted into an instruction style format for LLM training: e.g., User: <other's message> and Assistant: <user's original reply>. We load a base pre-trained model (e.g. Qwen 2.5 or Phi-3) and apply Low-Rank Adaptation (LoRA) to fine-tune on this data. LoRA updates only a small subset of parameters (≈1% of weights[6]), making training feasible on limited hardware and preserving general language ability. This fine-tuning should teach the model the user's unique language habits (e.g. word choices, emoticon use, code-mixing) and decision tendencies (i.e. how the user tends to answer certain questions).

- The training uses HuggingFace Transformers with PEFT: we set up an SFTTrainer, using Hyperparameters (batch size, learning rate, epochs) tuned for small data. For example, LoRA is applied to projection matrices, as recommended[5]. The dataset will be split (80/20) to monitor performance. We expect the fine-tuned model to capture stylistic features (like casual tone, emoji usage, frequent phrases) more robustly than retrieval prompts alone.

- **Integration & Inference:** Once trained, the LoRA adapter (plus base model) will be loaded into the chatbot code. A user's incoming message is then fed into the model with

a short prompt instructing it to reply as the user (e.g. "You are chatting as [Name]. Answer casually in Hindi+English mix."). At this stage, we may still use the RAG examples in the prompt for extra guidance, but the model itself now "knows" the user's profile.

- **Project Progress:** *So far*, data collection and preprocessing are complete, and an initial RAG prototype is functional (vector DB and retrieval code are working). We observed limitations in pure RAG replies (often repetitive or echoing the input), which aligns with known issues when queries are very short. These findings, along with research insights, guided our move to the LoRA fine-tuning strategy. (Indeed, LoRA and instruction fine-tuning have become popular for rapid customization of LLMs[5].) We are now training the model on the prepared dataset.

The overall **system architecture** thus consists of: *Telegram Bot → Message → Retrieval (optional) → LLM Prompt → LLM Response → Send Reply*. This pipeline leverages both memory (past chat examples) and the personalized language model to emulate the user's psychology and decision patterns.

---

# CHAPTER 3

## APPLICATIONS AND SCOPE

**Applications:**
- **Digital Assistant:** Acts as a conversational agent on behalf of the user. When the user is busy or offline, the bot can carry on chats with friends/family using learned speaking style.

- **Personal Messaging:** Helps users manage high chat volumes by automatically replying to routine messages (e.g. "I'm fine, you tell?") while still sounding like them.

- **Social Networking:** Could be extended to other platforms (Telegram, WhatsApp, etc.) so the user effectively has a **digital twin** on multiple messaging apps. This mirrors the user's personality and maintains social bonds even in absence.

- **Assistive Technology:** The twin can serve as a personal secretary, booking appointments or sending reminders in the user's tone. It might also aid users who have difficulty typing or communicating (e.g. due to disability), letting them "speak" through the AI assistant.

- **Research & Analytics:** The persona model could be used to study how language and decisions correlate in one's communication over time, combining insights from psychology with AI.

**Scope of the Project:**

- Currently, the project covers the complete pipeline from data to chatbot prototype. The main focus was on ensuring the chatbot's replies are linguistically and behaviorally similar to the real person.

- In the future, additional features can be added: multi-turn conversation memory (so the bot tracks context over a session), emotion recognition (for mood-appropriate replies), and real-time learning (updating the model as new messages come).

- The system could be integrated into a mobile app: messages sent to the user's WhatsApp number could automatically be handled by this assistant, providing immediate personalized responses.

- The digital twin concept could also extend beyond chat: for instance, it could assist in drafting emails or social media posts, always in the user's voice.

By combining data-driven modeling and advanced AI techniques, this project bridges human behavior and automated agents. If successful, it demonstrates how **behavioral adaptation** can be learned by an AI – a digital twin that not only speaks like the user, but "thinks" in a similar

way[1][4]. This represents a step towards truly personalized AI assistants, reflecting individual decision patterns and personality in everyday communication.

---

**References:**

Selected recent research underpins this work. For example, the **human digital twin** concept emphasizes modelling psychological traits[1], and personalized LLM-based chatbots have been shown to benefit from learning implicit user profiles[3]. LoRA fine-tuning is used following best practices for quick personalization[5][6]. These approaches combined form the technical foundation of our digital twin assistant.


[1] Towards human digital twin: Reviewing human modelling and simulation - ScienceDirect

https://www.sciencedirect.com/science/article/pii/S2452414X25001980

[2] TwinVoice: A Multi-dimensional BenchmarkTowards Digital Twins via LLM Persona Simulation

https://arxiv.org/html/2510.25536v1

[3] (PDF) One Chatbot Per Person: Creating Personalized Chatbots based on Implicit User Profiles

https://www.researchgate.net/publication/354088342_One_Chatbot_Per_Person_Creating_Personalized_Chatbots_based_on_Implicit_User_Profiles

[4] AI-Twin: AI-Powered Digital Twin for Personalized Assistance and Predictive Decision-Making – International Journal of Research and Scientific Innovation (IJRSI)

https://rsisinternational.org/journals/ijrsi/articles/ai-twin-ai-powered-digital-twin-for-personalized-assistance-and-predictive-decision-making/

[5] [6] LoRA can turn AI models into specialists quickly - IBM Research

https://research.ibm.com/blog/LoRAs-explained

[7] [2005.11401] Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks

https://arxiv.org/abs/2005.11401